1. Project ka main purpose

Humara project basically ek **Document Management + OCR + Search system** hai.

Requirement ye thi:

Ek directory mein thousands of documents hon, for example 5,000+, aur user kisi value jaise 1231232, invoice number, certificate number etc. search kare to system relevant documents return kare.

Documents 3 types ke ho sakte hain:

1. Normal text PDF

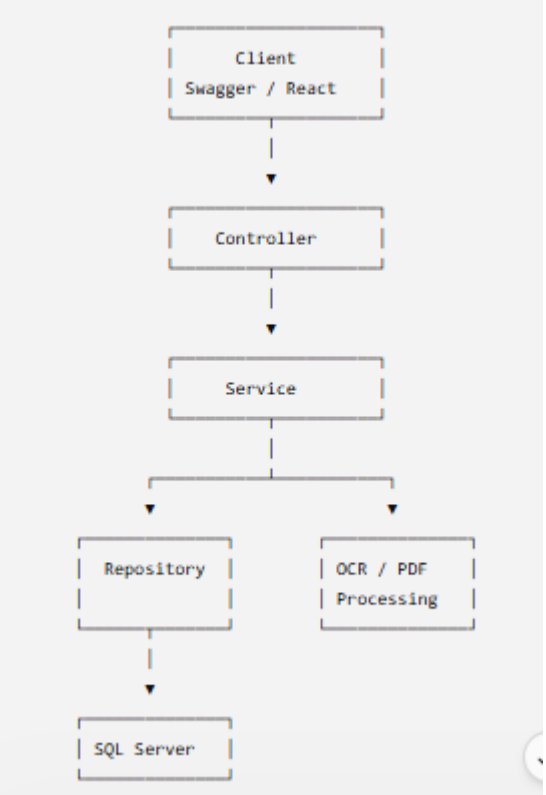
2. Scanned PDF

3. Images / scanned images

Problem ye hai ke scanned PDF/image ke andar actual selectable text nahi hota.

Isliye humne OCR add kiya.

Overall architecture:



---

2. ASP.NET Core Web API

Sabse pehle humne **ASP.NET Core Web API** project banaya.

Target:

.NET 9

Project ka naam:

DocumentSearch

Iska kaam API endpoints provide karna hai.

For example:

POST /api/Documents/upload

GET /api/Documents

GET /api/Documents/{id}

PUT /api/Documents/{id}

DELETE /api/Documents/{id}

Aur next step:

GET /api/Documents/search?q=1231232

---

3. SQL Server + Entity Framework Core

Humne documents ka metadata SQL Server mein store kiya.

Database:

DocumentSearch

Connection:

Server=.;

Database=DocumentSearch;

Trusted_Connection=True;

TrustServerCertificate=True;

Humne **Entity Framework Core** use kiya.

Libraries:

Microsoft.EntityFrameworkCore

Microsoft.EntityFrameworkCore.SqlServer

Microsoft.EntityFrameworkCore.Tools

EF Core ka role:

C# Model

↓

Entity Framework Core

↓

SQL Server

Yani humein manually har SQL query likhne ki zaroorat nahi.

---

4. Document Model

Humne Document entity banayi.

Conceptually:

Document

|

├── Id

├── UploadedByEmployeeId

├── FileName

├── ContentType

├── FilePath

├── FileSize

├── UploadedAt

└── ExtractedText

Sabse important property hai:

ExtractedText

Yahan document ke andar se nikala hua text store hota hai.

For example:

ExtractedText:

HALAL CERTIFICATE

Certificate No: NL10510500574

Awarded to: BRB International

...

Isi extracted text par future mein search chalegi.

---

5. Physical files SQL Server mein nahi rakhe

Ek important architectural decision:

Humne actual PDF/image ko database mein store nahi kiya.

Instead:

SQL Server

↓

metadata + file path + extracted text

File System

↓

actual PDF/image

Example:

Documents/

9d40354b-ab3e-4d90-8a1e-fba4c8591b62.pdf

Database mein:

FileName:

Application form for Halal Conformance (2).pdf

FilePath:

Documents\9d40354b-ab3e-4d90-8a1e-fba4c8591b62.pdf

Ye approach large document systems ke liye sensible hai.

---

6. GUID filename system

Agar user upload kare: invoice.pdf

Hum actual disk par: invoice.pdf nahi rakhte.

Instead: a8f4c3c1-....pdf

Yani:

var uniqueFileName = $"{Guid.NewGuid()}{extension}";

Iska faida:

- duplicate filename problem nahi
- existing file overwrite nahi hoti
- multiple users same filename upload kar sakte hain
- physical storage clean rehta hai

Original filename database mein preserve hota hai.

---

7. Repository Pattern

Humne directly Controller se EF Core nahi chalaya.

Architecture:

Controller

↓

Service

↓

Repository

↓

EF Core

↓

SQL Server

Repository:

IDocumentRepository

DocumentRepository

Repository ka kaam database operations hai.

For example:

GetAllAsync()

GetByIdAsync()

AddAsync()

UpdateAsync()

DeleteAsync()

Ye separation important hai because business logic aur database logic mix nahi hote.

---

8. Service Layer

Humne:

IDocumentService

DocumentService

banaya.

Service layer mein actual business logic hai.

For example upload ke waqt:

File receive karo

↓

File disk par save karo

↓

File type identify karo

↓

PDF?

↓

Text extraction / OCR

↓

ExtractedText

↓

Database mein metadata save

Ye Controller ke andar nahi rakha.

---

9. Upload API

Humne:

POST /api/Documents/upload

implement kiya.

Request:

multipart/form-data

Fields:

File

UploadedByEmployeeId

Example:

File = invoice.pdf

UploadedByEmployeeId = 4

System:

invoice.pdf

↓

Documents/{GUID}.pdf

↓

Text extraction

↓

SQL Server

---

10. PDF text extraction — PdfPig

Normal machine-readable PDFs ke liye humne:

PdfPig

use kiya.

Package:

<PackageReference Include="PdfPig" Version="0.1.16" />

Namespace:

UglyToad.PdfPig

Iska kaam:

PDF

↓

PDF pages

↓

actual selectable text

↓

string

Example:

CHECKLIST FOR HALAL CONFORMANCE

APPLICATION FORM FOR HALAL CONFORMANCE...

Humne test bhi kiya tha aur normal PDF ka text successfully ExtractedText mein aa gaya.

---

11. Problem: Scanned PDF

Yahan interesting part aaya.

Agar PDF scanned hai:

PDF

↓

image of document

to PdfPig ko actual text nahi milta.

Example:

PdfPig output:

SAMPLE LETTER

bas itna mila. Lekin actual scanned page par bohot saara text tha. Isliye humne OCR fallback banaya.

---

12. OCR — Tesseract

Images aur scanned PDFs ke liye humne:

Tesseract OCR

use kiya.

Package:

<PackageReference Include="Tesseract" Version="5.2.0" />

Aur:

tessdata/

  eng.traineddata

Tesseract ka basic flow:

Image

↓

Tesseract

↓

Recognized text

Example image:

HALAL CERTIFICATE

Awarded to: BRB International

Certificate No: NL105 10500574

OCR:

HALAL CERTIFICATE

Awarded to: BRB International

Certificate No: NL105 10500574

Kuch OCR mistakes normal hain.

---

13. Image OCR

Humne .jpg, .jpeg, .png ke liye direct OCR implement kiya.

Flow:

JPG / PNG

↓

Tesseract

↓

ExtractedText

↓

SQL Server

Humne Halal-Certificate.jpeg test kiya tha.

Result mein successfully:

HALAL CERTIFICATE

BRB International

Certificate No: NL105 10500574

...

extract hua.

So **image OCR successfully working hai.**

---

14. Scanned PDF ke liye PDF rendering

Tesseract directly PDF ko image ki tarah process nahi kar raha tha.

Isliye scanned PDF ke liye:

PDF

↓

Render each page as image

↓

PNG

↓

Tesseract

↓

Text

Humne iske liye:

PDFtoImage

use kiya.

Package:

<PackageReference Include="PDFtoImage" Version="5.4.0" />

Aur:

SkiaSharp

Package:

<PackageReference Include="SkiaSharp" Version="4.153.1" />

---

15. IPdfRenderer / PdfRenderer

Humne abstraction banayi:

IPdfRenderer

Implementation:

PdfRenderer

Iska kaam:

PDF

↓

Page 1 → page-1.png

Page 2 → page-2.png

Page 3 → page-3.png

...

Temporary folder mein images create hoti hain.

Example:

TempOCR/

  some-guid/

    page-1.png

page-2.png

page-3.png

Phir:

page-1.png

↓

Tesseract

page-2.png

↓

Tesseract

page-3.png

↓

Tesseract

Sab text combine ho jata hai.

---

16. Scanned PDF OCR fallback logic

Humne smart fallback rakha.

Pehle har PDF ko OCR nahi kar rahe.

Pehle:

PDF

↓

PdfPig

Agar enough text mil gaya:

Done ✅

Agar text: empty

ya bohot kam: < 20 characters

to OCR:

PDF

↓

PDFtoImage

↓

PNG pages

↓

Tesseract

↓

ExtractedText

Yani:

```
if (string.IsNullOrWhiteSpace(extractedText) ||
    extractedText.Trim().Length < 20)
{
    // OCR
}
```

Ye important optimization hai.

Har normal PDF ko unnecessarily OCR karne ki zaroorat nahi.

---

17. Scanned PDF ka actual successful test

Humne scanned pdf.pdf test kiya.

Pehli attempt mein PdfPig ne:

SAMPLE LETTER return kiya.

Humne threshold introduce ki:

< 20 characters → OCR

Phir same PDF dubara upload ki.

Tesseract ne successfully text extract kiya:

UNITED STATES ENVIRONMENTAL PROTECTION AGENCY

WASHINGTON, D.C. 20460

SAMPLE LETTER

OFFICE OF WATER

Dear Commissioner:

...

So humara complete OCR pipeline **successfully proven** hai.

---

18. Temporary OCR files automatically delete

OCR ke liye jo PNG files create hoti hain woh permanent nahi rakhi.

Flow:

Create TempOCR

↓

Render PDF pages

↓

OCR

↓

Extract text

↓

Delete TempOCR

Code mein finally use kiya hai taa-ke OCR ke baad cleanup ho.

Ye important hai warna thousands of PDFs process karte hue disk unnecessarily fill ho jayegi.

---

19. CRUD operations

Humne complete document CRUD bhi implement kiya.

Create

POST /api/Documents/upload

File upload + extraction + DB record.

Read all

GET /api/Documents

Read one

GET /api/Documents/{id}

Update

PUT /api/Documents/{id}

Actual physical file bhi replace hoti hai.

Old file:

delete

New file:

save

Aur extracted text bhi dobara generate hota hai.

Delete

DELETE /api/Documents/{id}

Database record aur physical file dono delete hote hain.

So basically **complete CRUD working hai.**

---

20. Employee relationship

Document ko employee ke saath associate kiya:

Document

↓

UploadedByEmployeeId

↓

Employee

Yani:

Employee ID = 4

ka matlab document kis employee ne upload kiya.

SQL Server mein foreign-key relationship use ki.

---

21. Swagger

Humne: Swashbuckle.AspNetCore use kiya.

Version: 9.0.6

Swagger se hum API ko browser mein directly test kar sakte hain.

For example:

POST upload

GET

PUT

DELETE

sab test kiye.

---

22. OpenAPI

Humne: Microsoft.AspNetCore.OpenApibhi use kiya.

Ye API documentation/OpenAPI support ke liye hai.

---

23. Dependency Injection

Humne ASP.NET Core ki built-in Dependency Injection use ki.

For example:

builder.Services.AddScoped<IDocumentRepository, DocumentRepository>();

builder.Services.AddScoped<IDocumentService, DocumentService>();

builder.Services.AddScoped<IOcrService, OcrService>();

builder.Services.AddScoped<IPdfRenderer, PdfRenderer>();

Aur service ko dependencies milti hain:

DocumentService

  |

  ├— DocumentRepository

  ├— PdfTextExtractor

  ├— OcrService

└── PdfRenderer

Isse code loosely coupled hai.

---

24. Humari libraries ka complete list

Current project mein important packages:

Library	Version	Purpose
Microsoft.EntityFrameworkCore	9.0.20	ORM / database access
Microsoft.EntityFrameworkCore.SqlServer	9.0.20	SQL Server support
Microsoft.EntityFrameworkCore.Tools	9.0.20	EF migrations/tools
PdfPig	0.1.16	Normal PDF text extraction
Tesseract	5.2.0	OCR
PDFtoImage	5.4.0	PDF pages → images
SkiaSharp	4.153.1	Image encoding/processing
Swashbuckle.AspNetCore	9.0.6	Swagger UI
Microsoft.AspNetCore.OpenApi	9.0.20	OpenAPI support

Ek aur package:

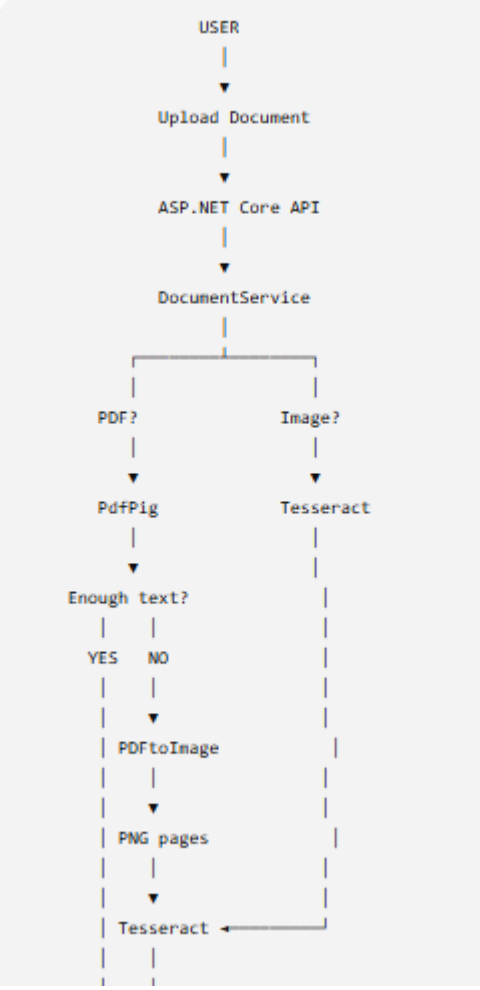
Docnet.Core 2.6.0

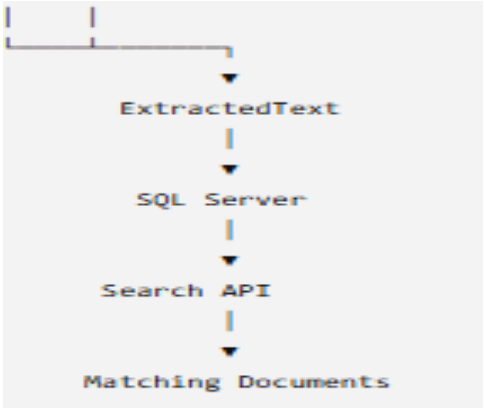
project mein abhi present hai, lekin **current implementation mein use nahi ho raha**. Humne PDF rendering ke liye PDFtoImage + SkiaSharp choose kiya hai.

---

25. Final complete pipeline

Ab humara system overall ye karta hai:





Aur sabse important point:

Hum **search ke waqt OCR nahi karenge**.

OCR sirf upload/update ke waqt:

Upload

↓

OCR

↓

ExtractedText save

Phir search:

User searches "1231232"

↓

SQL Server

↓

ExtractedText search

↓

Matching documents

Ye architecture 5,000 documents ke liye bilkul reasonable starting point hai.

Ab tak humne kya achieve kiya?

Phase 1 — Document management: ✅
Upload, Get, Update, Delete

Phase 2 — File storage: ✅
Physical files + unique GUID names

Phase 3 — Database: ✅
SQL Server + EF Core

Phase 4 — Normal PDF extraction: ✅
PdfPig

Phase 5 — Image OCR: ✅
Tesseract

Phase 6 — Scanned PDF OCR: ✅
PDFtoImage → SkiaSharp → Tesseract

Phase 7 — Temporary OCR cleanup: ✅

Phase 8 — Search: 🔜
GET /api/Documents/search?q=1231232

So honestly, ab project sirf **simple CRUD API** nahi raha. Ye ek proper **document-processing backend** ban gaya hai jisme file management, PDF parsing, OCR, database persistence aur eventually full-text search combine ho rahe hain.